

Programación de Acciones Robóticas

Robin Omar Buezo Díaz



Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

Indice

Contenido

Indice	1
Competencia(s)	2
Definición de trayectorias y movimientos.....	2
Objetivo.....	2
Introducción.....	2
Contenido	2
Pongamos en Práctica la Teoría	3
Caso de Estudio 1.....	3
Caso de Estudio 2.....	3
Conclusiones	3
Respuesta a eventos en simulaciones robóticas	3
Objetivo.....	3
Introducción.....	3
Contenido	3
Pongamos en Práctica la Teoría	4
Caso de Estudio 1.....	4
Caso de Estudio 2.....	4
Conclusión.....	4
Conclusión General	4
Referencias	4

Competencia(s)

Integra diseños 2D y 3D de robots utilizando librerías como p5.js y Three.js en simulaciones programadas que involucren acciones y comportamientos dinámicos.

Definición de trayectorias y movimientos

Objetivo

Desarrollar algoritmos que controlen desplazamientos, direcciones y movimientos estructurados en robots simulados o físicos.

Introducción

Programar un robot implica definir cómo se mueve en el espacio, con qué velocidad, por qué trayecto y en qué momento. Esto incluye movimientos lineales, giros, trayectorias predeterminadas o adaptativas. Estos comportamientos pueden controlarse desde código o mediante herramientas gráficas de simulación.

Contenido

Elementos clave en la definición de movimientos:

- **Coordenadas:** posición (x, y, z)
- **Orientación:** ángulo o vector de dirección
- **Velocidad lineal y angular**
- **Tiempo de ejecución**
- **Condiciones de inicio o parada**

Tipos de trayectorias:

- Rectas simples
- Curvas o circulares
- Rutas programadas (puntos de control)
- Trayectorias generadas por sensores o algoritmos

Motores de simulación como Webots, CoppeliaSim o Unity permiten controlar movimientos con scripts en Python, Lua, JavaScript o C++.

Pongamos en Práctica la Teoría

Caso de Estudio 1

En un simulador, se programa un robot móvil para avanzar 2 metros, girar 90 grados, y repetir el ciclo. Se usa una función que combina tiempo y velocidad para definir la trayectoria cuadrada.

Caso de Estudio 2

Un brazo robótico simulado se programa para mover su pinza desde el punto A hasta B pasando por una trayectoria curva, utilizando interpolación entre coordenadas y control de rotación conjunta.

Conclusiones

La planificación de trayectorias convierte un modelo robótico en un agente funcional. Dominar esta técnica permite aplicar conceptos físicos, espaciales y algorítmicos en tareas complejas.

Respuesta a eventos en simulaciones robóticas

Objetivo

Programar respuestas automáticas ante condiciones detectadas por sensores o eventos externos durante una simulación.

Introducción

Un robot autónomo no solo se mueve, también percibe su entorno y toma decisiones. En simulaciones, esto se reproduce mediante eventos lógicos como detección de colisiones, cambios en variables o entrada de datos.

Contenido

Eventos comunes en simulación:

- **Colisión con objeto**
- **Sensor detecta obstáculo**
- **Cambio de estado o variable**
- **Interacción del usuario (teclado, mouse, voz)**

Estrategias de respuesta:

- Condicionales (`if`, `switch`)
- Mapeo de eventos → acciones
- Subrutinas de evasión o corrección
- Uso de estados (espera, alerta, activo)

Herramientas como Webots o Unity usan funciones como `onCollisionEnter()`, `sensorInput()`, o eventos personalizados para manejar estas reacciones.

Pongamos en Práctica la Teoría

Caso de Estudio 1

Un robot aspiradora simulado cambia de dirección si su sensor frontal detecta un obstáculo. El evento activa una rutina de retroceso y giro aleatorio.

Caso de Estudio 2

En una simulación educativa, el robot reacciona a la tecla "R" ejecutando una secuencia programada de movimientos. Esta respuesta permite controlar su modo manual o autónomo.

Conclusión

Responder a eventos permite que el robot interactúe con su entorno. Estas reacciones simulan inteligencia básica y permiten validar decisiones de control en tiempo real.

Conclusión General

Programar acciones robóticas implica combinar control de movimiento y reacciones condicionales. Esto transforma un modelo inerte en un agente autónomo capaz de navegar, interactuar y adaptarse dentro de su entorno simulado o real.

Referencias

Michel, O. (2004). *Webots: Professional Mobile Robot Simulation*
Coppelia Robotics. (2023). *CoppeliaSim Documentation*
Siciliano, B., & Khatib, O. (2016). *Springer Handbook of Robotics*
Unity Robotics Hub – Unity Docs (2023)